Exp 14: Exploring Pro-Code Extension of a Zoho Creator Application using Zoho Catalyst / Custom Functions

Name: Suriya S | **Roll No:** 241401112 | Dept. of Computer Science and Engineering, Rajalakshmi Engineering College

1. Aim

To extend the Gym Membership Enrollment and Front Desk Management application in Zoho Creator beyond low-code workflows by writing Deluge custom functions and by calling a serverless function hosted on Zoho Catalyst, so that the membership end date and the total fee are calculated automatically when an enrolment is submitted.

2. Procedure

2.1 Pro-Code Requirements

Until Exp 12 the front desk typed the fee and counted the membership end date by hand, which leads to mistakes and different rules between staff. Four requirements were handled, as listed in Table 1.

Requirement	Component	Details
Calculate the end date	Deluge custom function	calculateEndDate(startDate, months) returns the last day of the membership
Calculate the total fee	Catalyst Advanced I/O function	gym_quote applies the plan rate, family discount and tax and returns a JSON quote
Connect Creator to Catalyst	Connection and invokeurl	A Deluge function posts the enrolment details to Catalyst and reads the reply
Run it automatically	Form workflow	On Success of Membership_Enrollment_Form, both functions run and update the record

Table 1: Pro-code requirements and the components used

2.2 Choosing Between Low-Code, Custom Functions and Catalyst

Creator's drag-and-drop workflows handle most form logic. Custom functions are used when the same logic is needed in several places, and Catalyst is used when the logic is better kept outside Creator, for example in a general-purpose language with its own testing and versioning. Table 2 records the comparison and where each was applied.

Option	Best suited for	Used in this application
Low-code workflow	Simple field updates, emails, validations	Form workflow that triggers the functions
Deluge custom function	Reusable logic inside Creator	calculateEndDate and getQuote
Zoho Catalyst function	Pro-code logic in Node.js, Java or Python, kept outside Creator	gym_quote fee calculation

Table 2: Extension options compared

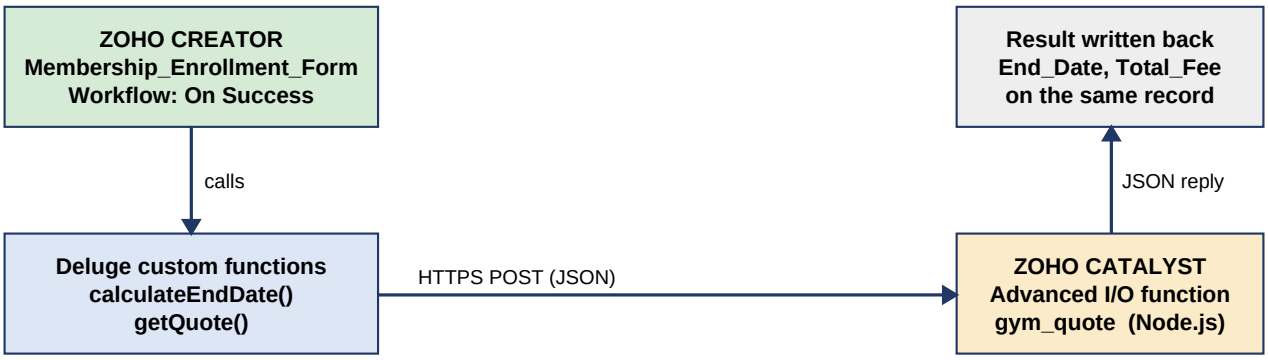

Figure 1: Architecture of the pro-code extension (illustrative diagram)

2.3 Writing the Custom Functions in Creator

In the Creator builder, a function was created under Workflows > Functions for each task. The first function, shown in Listing 1, takes the start date and number of months and returns the end date.

Listing 1: calculateEndDate (Deluge custom function)

```
date calculateEndDate(date startDate, int months)
{
    // membership ends one day before the same date after N months
    endDate = startDate.addMonth(months).subDay(1);
    return endDate;
}
```

2.4 Building the Catalyst Function

A Catalyst project was created and an Advanced I/O function named gym_quote was added in Node.js. It exposes a POST route /quote. The plan rate, family discount (5% per family member, capped at 15%) and tax (set as a constant) are kept in this one place, so a change in pricing needs one edit and one deployment, not changes across forms. The function was tested in the development environment of Catalyst before use.

Listing 2: gym_quote Advanced I/O function (Node.js)

```
const express = require("express");
const app = express();
app.use(express.json());

const RATE = { Basic: 1000, Standard: 1500, Premium: 2500 };  // fee per month
const GST = 0.18;                                             // tax rate constant
const round2 = (n) => Math.round(n * 100) / 100;

app.post("/quote", (req, res) => {
  const { plan, months, family_count } = req.body;
  if (!RATE[plan] || !(months > 0)) {
    return res.status(400).json({ error: "invalid input" });
  }
  const subtotal = RATE[plan] * months;
  const pct = Math.min((family_count || 0) * 5, 15);   // 5% per member, max 15%
  const discount = round2(subtotal * pct / 100);
  const taxable = round2(subtotal - discount);
  const gst = round2(taxable * GST);
  res.json({ subtotal, discount_percent: pct, discount, taxable, gst,
             total: round2(taxable + gst) });
});

module.exports = app;
```

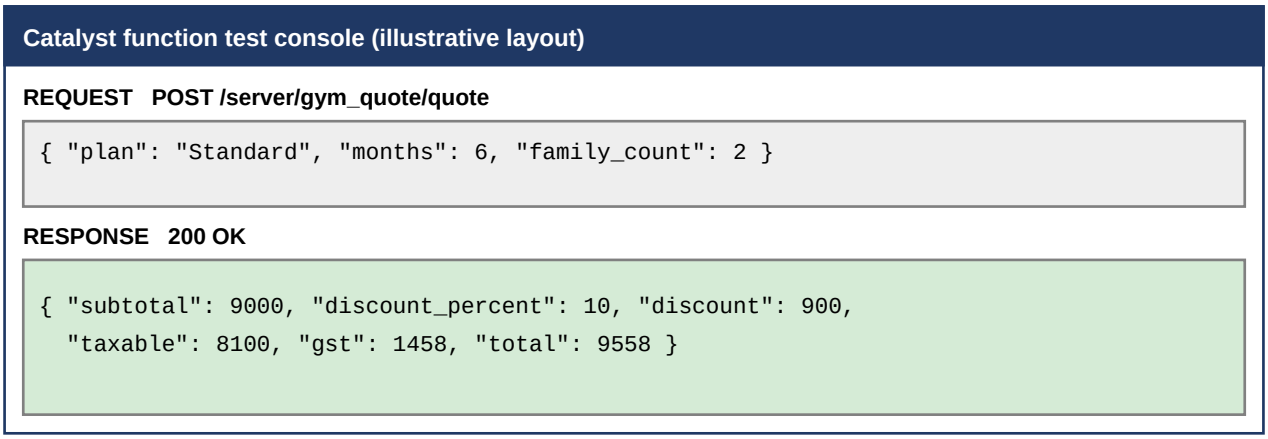

Figure 2: Sample request and response for the gym_quote function (illustrative diagram)

2.5 Connecting Creator to Catalyst

A Zoho OAuth connection named catalyst_conn was created under Settings > Connections in Creator, so no password or token appears in the code. The second function, getQuote (Listing 3), sends the plan, duration and family-member count to the Catalyst endpoint and returns the reply. The call is wrapped in try-catch, so a Catalyst outage does not block enrolments.

Listing 3: getQuote (Deluge custom function calling Catalyst)

```
map getQuote(string plan, int months, int familyCount)
{
    payload = Map();
    payload.put("plan", plan);
    payload.put("months", months);
    payload.put("family_count", familyCount);
    result = Map();
    try
    {
        response = invokeurl
        [
            url :"https://<project-domain>/server/gym_quote/quote"
            type :POST
            parameters:payload.toString()
            headers:{"Content-Type":"application/json"}
            connection:"catalyst_conn"
        ];
        result = response;
    }
    catch (e)
    {
        info "Catalyst call failed: " + e;
        result.put("error", true);
    }
    return result;
}
```

2.6 Calling the Functions from a Form Workflow

A workflow on Membership_Enrollment_Form, running On Success of Create, calls both functions and writes the results back to the record (Listing 4). Figure 3 shows the sequence.

Listing 4: Form workflow using the custom functions

```
// Form: Membership_Enrollment_Form    Event: On Success (Create)
input.End_Date = thisapp.calculateEndDate(input.Start_Date, input.Duration_Months);

quote = thisapp.getQuote(input.Membership_Plan, input.Duration_Months, input.Family_Members.count());
if(quote.get("error") != true)
{
    input.Total_Fee = quote.get("total");
}
```

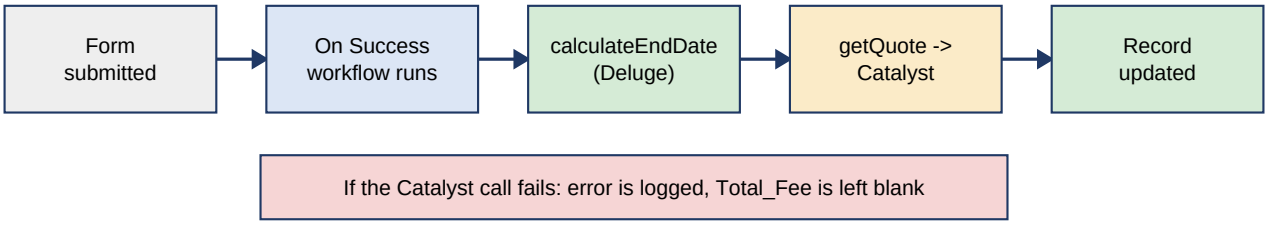

Figure 3: Sequence from form submission to updated record (illustrative diagram)

2.7 Testing

Sample enrolments were submitted with a start date of 1 November 2026 and the results compared with a hand calculation (Table 3). Each row was checked against the Total_Fee and End_Date saved on the record.

Plan	Months	Family members	Expected end date	Expected total fee (Rs.)
Standard	6	2	30 Apr 2027	9,558.00
Basic	3	0	31 Jan 2027	3,540.00
Premium	12	4 (discount capped at 15%)	31 Oct 2027	30,090.00

Table 3: Test cases with hand-calculated expected values (sample data)

Figure 4: Effect of the extension on the enrolment process (illustrative diagram)

2.8 Governance and Good Practice

The practices from Exp 12 were carried over to the pro-code parts, as listed in Table 4.

Practice	How it is applied
No secrets in code	Authentication is held in the catalyst_conn connection, not typed into the function
Data minimisation	Only plan, duration and family-member count are sent to Catalyst; health notes and certificates are not
Error handling	try-catch in getQuote; the enrolment is still saved if Catalyst is unreachable
Controlled releases	Catalyst function tested in development first; Creator changes tested before deploying to production (Exp 11)
Single source of rules	Pricing rules live only in gym_quote, so they are changed in one place

Table 4: Governance practices for the pro-code extension

3. Output

The listings show the two custom functions, the Catalyst function and the workflow, and the figures illustrate the architecture, the request and response, and the sequence of the automated process. The test cases in Table 3 give the values the saved records should show.

Observations from the output

- Custom functions make Creator logic reusable, so the same end-date rule can be called from any form or report.
- Catalyst suits logic that changes often or needs a full programming language, because it can be edited and tested outside Creator.
- Wrapping the Catalyst call in try-catch keeps the front desk working even when the external service fails.
- Using a connection instead of a pasted token keeps credentials out of the code and out of the exported application.
- Sending only the fields the quote needs follows the data-minimisation habit set in Exp 12.
- Hand-calculated test cases make it easy to confirm that the function and the workflow agree.

4. Result

The Gym Membership Enrollment and Front Desk Management application was extended with pro-code components. Two Deluge custom functions and a Zoho Catalyst Advanced I/O function were written, connected through a Zoho OAuth connection, and called from a form workflow so that the membership end date and the total fee are filled automatically on each enrolment. This completes the pro-code extension requirements of this experiment.